
Security Review Report

NM-0928 - STABILIZER



NETHERMIND
SECURITY

(August 18, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[High] A collateral depeg flows straight into USDZ	6
6.2	[High] Asset-side LP fees round to zero and become permanently unclaimable in low-decimal pools	6
6.3	[High] Depeg protection is enforced only on the pool swap path, not on USDZ mint and redeem	7
6.4	[Medium] Adding liquidity mints LP shares at face value and can dilute existing liquidity providers	7
6.5	[Low] Pool swaps have no slippage protection, exposing direct callers to forced slippage	8
6.6	[Low] Mint and redeem fees are global and cannot be set per asset	9
6.7	[Low] The redeem fee accrues as reserve surplus and can be locked below the k/2 floor	9
6.8	[Low] Missing lock period lets an LP front-run a large swap and capture its fees risk-free	10
6.9	[Low] Redeem fees aren't tracked and affect adapter solvency checks	10
6.10	[Info] The AdapterUSDS contract overrides balance() to return the live token balance, diverging from the other adapters	11
6.11	[Info] The guardian can halt pool's swaps through the uncapped emergency fee	12
6.12	[Info] The AMM and pool pause switches are misaligned in both directions	12
6.13	[Info] Redeeming a small USDZ amount burns USDZ but returns zero asset for low-decimal collateral	12
6.14	[Info] Unnecessary transfer before burn during redeem	13
6.15	[Best Practices] The setGuardian(...) function emits the wrong value for the old guardian	13
6.16	[Best Practices] Unused function updateReserve	14
6.17	[Best Practices] Unnecessary double declaration of pool asset	14
6.18	[Best Practices] The USDZ contract should inherit Openzeppelin ERC20 logic	14
7	Documentation Evaluation	16
8	Test Suite Evaluation	17
8.1	Tests Output	17
9	About Nethermind	18

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for Stabilizer. The focus of the engagement has been the Stabilizer AMM, ReserveHolder, Pool, and Adapter contracts, enabling zero-slippage stablecoin swaps on approved assets.

The protocol enables users to exchange approved stablecoin assets for USDZ, which can then be used in other DeFi applications or provided as liquidity alongside another approved asset in Stabilizer pools. These pools enable 1:1 swaps between USDZ and other stablecoins with zero slippage and a small, customizable fee. The pools operate using a constant-sum mechanism, with funds deposited into or withdrawn from an adapter contract through rebalancing to keep each pool balanced.

The audit comprises 1521 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 18 points of attention, where 3 are classified as High. 1 is classified as Medium. 5 are classified as Low. and 9 are classified as Informational or Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

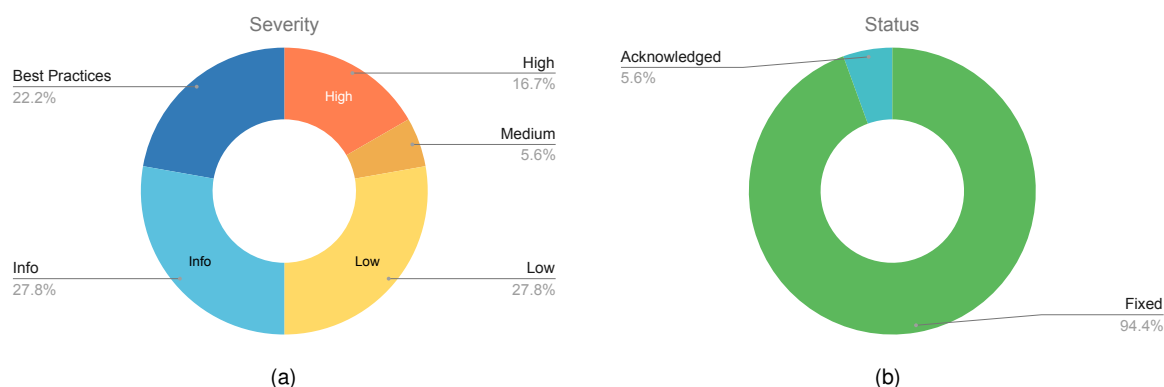


Fig. 1: Distribution of issues: Critical (0), High (3), Medium (1), Low (5), Undetermined (0), Informational (5), Best Practices (4).
Distribution of status: Fixed (17), Acknowledged (1), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	July 27, 2026
Final Report	August 18, 2026
Initial Commit	d13226947bc23461937881f29eb0a4233136b301
Final Commit	c4caafae4f50f4cf9fe37c4ce64d5806882c5701
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	contracts/StabilizerRouter.sol	119	40	33.6%	39	198
2	contracts/StabilizerAMM.sol	228	10	4.4%	46	284
3	contracts/ReserveHolder.sol	144	7	4.9%	29	180
4	contracts/USDZ.sol	111	34	30.6%	36	181
5	contracts/pools/SPool.sol	24	7	29.2%	4	35
6	contracts/pools/PPool.sol	24	11	45.8%	4	39
7	contracts/pools/TPool.sol	24	7	29.2%	4	35
8	contracts/pools/CPool.sol	24	7	29.2%	4	35
9	contracts/pools/PoolBase.sol	373	10	2.7%	90	473
10	contracts/libraries/StableSwapMath.sol	144	27	18.8%	20	191
11	contracts/adapters/AdapterPYUSD.sol	14	12	85.7%	4	30
12	contracts/adapters/AdapterBase.sol	61	6	9.8%	16	83
13	contracts/adapters/AdapterUSDS.sol	17	8	47.1%	5	30
14	contracts/adapters/AdapterUSDC.sol	14	8	57.1%	4	26
15	contracts/adapters/AdapterUSDT.sol	14	8	57.1%	4	26
16	contracts/helpers/Proxy.sol	2	2	100.0%	1	5
17	contracts/interfaces/IStabilizerAMM.sol	37	8	21.6%	7	52
18	contracts/interfaces/IUSDZ.sol	15	3	20.0%	4	22
19	contracts/interfaces/IPool.sol	55	27	49.1%	11	93
20	contracts/interfaces/IStabilizerRouter.sol	36	3	8.3%	6	45
21	contracts/interfaces/IReserveHolder.sol	28	3	10.7%	3	34
22	contracts/interfaces/IAdapter.sol	13	2	15.4%	2	17
	Total	1521	250	16.4%	343	2114

3 Summary of Issues

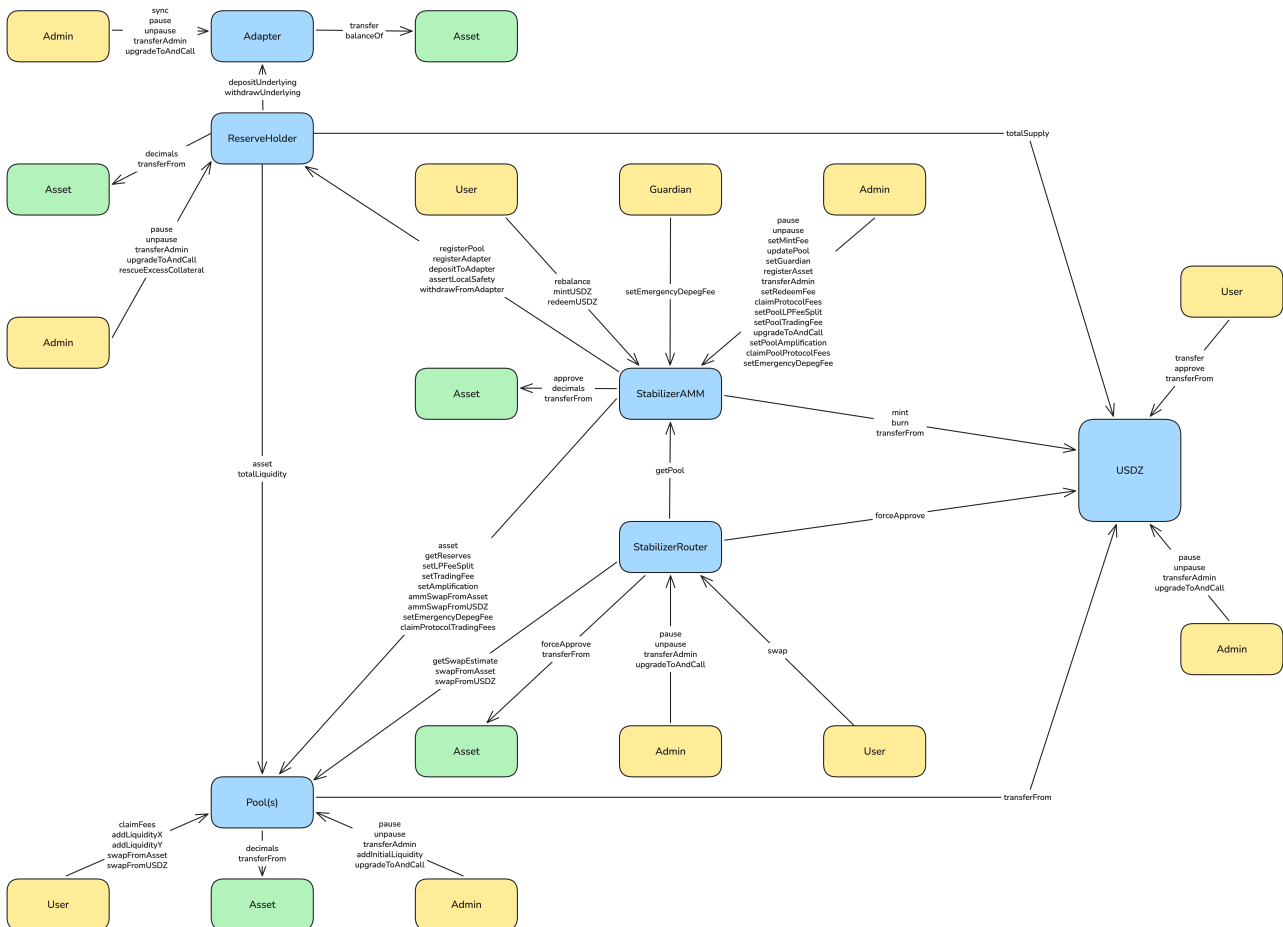
	Finding	Severity	Update
1	A collateral depeg flows straight into USDZ	High	Fixed
2	Asset-side LP fees round to zero and become permanently unclaimable in low-decimal pools	High	Fixed
3	Depeg protection is enforced only on the pool swap path, not on USDZ mint and redeem	High	Fixed
4	Adding liquidity mints LP shares at face value and can dilute existing liquidity providers	Medium	Fixed
5	Pool swaps have no slippage protection, exposing direct callers to forced slippage	Low	Fixed
6	Mint and redeem fees are global and cannot be set per asset	Low	Fixed
7	The redeem fee accrues as reserve surplus and can be locked below the k/2 floor	Low	Fixed
8	Missing lock period lets an LP front-run a large swap and capture its fees risk-free	Low	Fixed
9	Redeem fees aren't tracked and affect adapter solvency checks	Low	Fixed
10	The AdapterUSDS contract overrides balance() to return the live token balance, diverging from the other adapters	Info	Fixed
11	The guardian can halt pool's swaps through the uncapped emergency fee	Info	Acknowledged
12	The AMM and pool pause switches are misaligned in both directions	Info	Fixed
13	Redeeming a small USDZ amount burns USDZ but returns zero asset for low-decimal collateral	Info	Fixed
14	Unnecessary transfer before burn during redeem	Info	Fixed
15	The setGuardian(...) function emits the wrong value for the old guardian	Best Practices	Fixed
16	Unused function updateReserve	Best Practices	Fixed
17	Unnecessary double declaration of pool asset	Best Practices	Fixed
18	The USDZ contract should inherit Openzeppelin ERC20 logic	Best Practices	Fixed

4 System Overview

The Stabilizer system is a collateral-backed stablecoin protocol that coordinates mint/redeem flows, pooled liquidity, and reserve accounting to keep USDZ redeemable and swaps liquid. Users can mint USDZ with supported collateral, redeem it for collateral, provide two-sided pool liquidity, and swap between collateral and USDZ through a hybrid pricing curve that adapts during imbalance or depeg pressure. Reserves are tracked in normalized units, pool safety constraints are enforced before sensitive outflows, and protocol and LP fees are accounted for separately. The system consists of four main components: the StabilizerAMM, ReserveHolder, Adapters, and Pools. At the end of this section a diagram presenting the call interactions between all contracts is shown.

- **StabilizerAMM (StabilizerAMM.sol)**: The protocol coordinator for user-facing mint/redeem operations, asset routing, and pool rebalancing. It converts collateral deposits into USDZ using decimal normalization and protocol mint fees, then routes the collateral into the reserve layer. During redemptions, it burns USDZ and releases collateral subject to reserve safety checks. It also measures pool imbalance and executes corrective swaps to move pools toward their target balance, while exposing configuration options for fees and pool parameters.
- **ReserveHolder (ReserveHolder.sol)**: The system balance sheet that tracks collateral backing USDZ across supported assets. It records per-asset reserves, aggregates normalized reserve values, compares them against circulating USDZ, and enforces global solvency and pool-local safety thresholds before withdrawals. Inflows increase tracked collateral, while outflows reduce it only when the relevant constraints remain satisfied. Adapter and pool mappings identify where assets are managed and which risk boundaries apply.
- **Adapters (AdapterBase.sol)**: The collateral execution layer that abstracts how and where underlying assets are held while exposing a uniform deposit and withdrawal interface. The base adapter maintains an internal balance ledger, accepts reserve-directed deposits, and processes withdrawals to recipients. This separates reserve accounting from token-specific handling and allows new collateral integrations to be added without changing the AMM or reserve logic.
- **Pools (PoolBase.sol)**: The liquidity and price-discovery layer where collateral and USDZ are paired for liquidity provision and swaps. Pools maintain dual reserves, mint and burn LP shares, apply trading fees and emergency or depeg overrides, and calculate swap outputs using hybrid stable-swap math that adapts to balanced and imbalanced states. Trading fees are split between LPs and the protocol through per-share accrual accounting, supporting predictable claims while preserving liquidity.

Figure 1: Protocol Interaction Diagram



Remarks about Stabilizer's oracle-less design

The Stabilizer protocol is designed to facilitate 1:1 swaps between stablecoins, falling back to the StableSwap pricing model when a 1:1 exchange is not feasible. This design allows swaps to be executed without relying on an external price oracle during normal protocol operation.

However, the absence of an onchain price oracle means that the protocol cannot independently determine whether a supported asset has depegged from its intended value. This limitation affects not only swaps, but also the protocol's mint and redeem flows. Unless the relevant pool is paused or appropriate fees are applied, users may continue to mint, redeem, or swap assets using protocol-defined pricing that no longer reflects their external market value.

To manage this risk, Stabilizer relies on offchain infrastructure to monitor the market prices of supported assets and respond to depeg events. Upon detecting a depeg, this infrastructure is expected to pause the affected pool or set appropriate emergency fees. The effectiveness of these protections therefore depends on the offchain systems detecting the event accurately and responding sufficiently quickly.

Even where emergency fees are updated at regular intervals, the market price of a depegged asset may move significantly between updates. This may allow opportunistic traders or arbitrageurs to mint, redeem, or swap against the protocol at favorable terms before the fees are adjusted or the affected pool is paused.

The behavior and reliability of this offchain infrastructure cannot be verified onchain, and its implementation was not reviewed as part of this engagement. Although the protocol is oracleless during normal operation, its ability to respond safely to a depeg depends on centralized offchain systems and authorized actors operating correctly and rapidly. A failure, delay, or misconfiguration in these systems could leave the protocol exposed during periods of significant market volatility.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [High] A collateral depeg flows straight into USDZ

File(s): [contracts/StabilizerAMM.sol](#)

Description: USDZ is minted against collateral at a fixed nominal 1:1, with no price feed. The `mintUSDZ(...)` function treats one unit of any supported asset as one dollar of backing and mints USDZ accordingly, and `isSolvent()` in `ReserveHolder` measures backing by token quantity rather than market value. This is by design, since the protocol is oracle-free, and it holds while every collateral trades at \$1.

The problem appears when a collateral asset loses its peg. If USDC falls to \$0.90, `mintUSDZ(...)` still mints one USDZ per USDC, so every mint injects \$0.10 of bad debt into USDZ's backing, and nothing in the mint path prices the asset any lower. The only knob the protocol has here is the mint fee, and it cannot defend the peg.

The fee is taken out of the collateral pool rather than kept as backing. `mintUSDZ(...)` forwards only the post-fee amount into the reserve and books the fee as protocol revenue:

```
IReserveHolder(reserveHolder).depositToAdapter(address(this), asset, amountAssetIn - fee);
// @audit-issue The fee is booked as revenue, so it never backs USDZ.
accumulatedFees[asset] += fee;
IUSDZ(usdz).mint(msg.sender, amountUSDZOut);
```

The minted USDZ always equals the collateral added (both are `amountAssetIn - fee`), so the backing ratio stays at a nominal 1:1 no matter how high the fee is set; raising the fee just shrinks both sides together. Suppose USDC is at \$0.90 and the fee is raised to 10% to match. A 1 USDC mint deposits 0.9 USDC into the reserve and mints 0.9 USDZ, so the new USDZ is backed by \$0.81, still only \$0.90 per USDZ.

Recommendation(s): Consider implementing a special haircut for depeg scenarios that can feed into USDZ reserves.

Status: Fixed

Update from the client: resolved in commit: [6637a1cc424df517967e0f4a4c8a62efcb41a130](#)

6.2 [High] Asset-side LP fees round to zero and become permanently unclaimable in low-decimal pools

File(s): [contracts/pools/PoolBase.sol](#)

Description: Every pool pays trading fees to its liquidity providers with a per-share accumulator. Two terms matter here. `totalSupply` is the pool's LP-share supply, the total liquidity provider tokens minted for this pool, which changes only when someone adds or removes liquidity. `accFeePerShareX` is a running total of "fee earned per LP share, summed over all swaps"; it exists so the pool can pay many LPs without looping over them, since any LP's claim is just their shares times how much the accumulator grew while they held those shares.

On an asset-side swap the LP part of the fee is added to `feeReserveX`, the pot LPs later withdraw from, and `accFeePerShareX` is bumped so that new fee becomes claimable. An LP's claimable amount is read straight off that accumulator, so if it does not move, the fee cannot be claimed.

```
function _distributeFeeX(uint256 feeAmount) internal {
    // ...
    if (lpFee > 0) {
        feeReserveX += lpFee;
        // @audit-issue Rounds to 0 when lpFee < totalSupply / 1e18, and the remainder is dropped.
        if (totalSupply > 0) accFeePerShareX += (lpFee * ACC_PRECISION) / totalSupply;
    }
    // ...
}
```

The two lines use different units. `totalSupply` is tracked on an 18-decimal basis, but the asset-side `lpFee` is in the asset's own decimals. For a 6-decimal asset like USDC that is a 12-decimal gap, and `ACC_PRECISION` is `1e18`. So the increment `lpFee * 1e18 / totalSupply` rounds down to 0 whenever `lpFee` is smaller than `totalSupply / 1e18`. The leftover is not saved for next time, so a run of small fees never adds up either: `feeReserveX` keeps growing while the accumulator stays put.

The cutoff is a fixed fraction of pool depth. On a pool holding 1,000,000 per side, `totalSupply` is `2e24`, so the accumulator only moves once `lpFee` reaches 2 USDC, which at the default 1 bps fee is any `swapFromAsset(...)` below 20,000 USDC. That is about 2% of the per-side size, and the same 2% rule holds at any pool size. Since most swaps are far smaller than 2% of the pool, ordinary trading strands its asset-side fee almost every time, and the 1 bps rate the protocol advertises is the worst case.

When the accumulator does not move, no LP accrues anything, so the fee sitting in `feeReserveX` is unreachable. The protocol cannot take it either, because it was routed to `feeReserveX` and not `protocolFeeReserveX`, and `removeLiquidity(...)` pays out only `reserveX` and `reserveY`. There is no path that pays out `feeReserveX` beyond what the accumulator recorded, so the stranded fee is frozen in the pool for good.

This affects the asset side only. The USDZ side is already 18-decimal, so its cutoff is below one wei and it credits normally.

Recommendation(s): Consider matching the units before the division, for example by scaling the asset-side fee up to the same 18-decimal basis as `totalSupply`, or by using a much larger accumulator precision, so a realistic fee no longer rounds to zero.

Status: Fixed

Update from the client: resolved in commit: [19c2a092a57becac0d19a58dc8a0434c8f8c3483](#)

6.3 [High] Depeg protection is enforced only on the pool swap path, not on USDZ mint and redeem

File(s): `contracts/StabilizerAMM.sol`

Description: The protocol has two separate ways to convert between a collateral asset and USDZ. One is a pool swap (`swapFromAsset(...)` / `swapFromUSDZ(...)`), which is priced on the hybrid curve and carries both the trading fee and the emergency depeg fee. The other is minting and redeeming on the StabilizerAMM contract (`mintUSDZ(...)` / `redeemUSDZ(...)`), which converts at a fixed 1:1 rate and carries only a small mint or redeem fee.

The emergency depeg fee is the protocol's defense during a depeg. When an asset loses its peg the guardian raises that pool's fee through `setEmergencyDepegFee(...)`, making it expensive to trade the bad asset. The fee is read by `getEffectiveFee(...)` and charged on both directions of a pool swap:

```
function swapFromAsset(uint256 amountIn) external ... {
    // ...
    uint256 feeAmount = (amountIn * getEffectiveFee()) / BPS_DENOMINATOR;
    uint256 netInput = amountIn - feeAmount;
    // ...
}
```

The root problem is that this defense exists only on the pool path. The mint and redeem path, which is the more direct way to move value between an asset and USDZ, never reads the emergency fee. It charges only `mintFeeBPS` / `redeemFeeBPS`, which default to 1 BPS and are set by the admin, not the guardian:

```
function mintUSDZ(address asset, uint256 amountAssetIn) external ... {
    // ...
    // @audit-issue Only mintFeeBPS is charged here; the pool emergency fee is never read.
    uint256 fee = (amountAssetIn * mintFeeBPS) / BPS_DENOMINATOR;
    uint256 amountAssetAfterFee = amountAssetIn - fee;
    amountUSDZOut = amountAssetAfterFee * (10 ** (18 - assetDecimals));
    // ...
}
```

So any protection the guardian applies to a pool can be sidestepped by routing through mint or redeem instead. The clearest impact is that the emergency fee can be bypassed outright. Suppose USDC drops to \$0.90 and the guardian sets a large emergency fee on the USDC pool. An attacker holding cheap USDC skips the pool and calls `mintUSDZ(...)`, turning USDC worth \$0.90 into USDZ worth \$1 for a 1 BPS fee, then swaps that USDZ into a healthy asset such as USDT or simply holds it. The protocol is left holding the bad USDC as backing, and the loss falls on the protocol and its LPs, which is exactly what the emergency fee was meant to stop. This was reproduced in a PoC: with a 50% emergency fee live on the USDC pool, the same USDC produces almost no USDZ through the pool but a near full 1:1 amount through `mintUSDZ(...)`, and the full USDC → USDZ → USDT route lets the attacker exit close to par.

The same root cause shows up in several other ways. The guardian's emergency lever has no reach over the main path, since `setEmergencyDepegFee(...)` only touches pools while mint and redeem can be stopped only by `pause()`, which is `onlyAdmin`, so the fast, per-asset, lower-trust response the protocol built for a depeg does not cover the path an attacker would actually use, and only a full admin pause does. The redeem side makes this worse: once an asset depegs USDZ becomes partly backed by the bad asset, yet `redeemUSDZ(...)` still pays out a healthy asset at 1:1 (minus the redeem fee), letting holders race to pull good collateral and leave the bad behind while the emergency fee cannot slow them down, because redeem never reads it.

Recommendation(s): Consider enforcing the depeg protection on every path that converts between an asset and USDZ, not just pool swaps, for example by applying the same per-asset, guardian-settable brake (or a shared peg check) inside `mintUSDZ(...)` and `redeemUSDZ(...)`.

Status: Fixed

Update from the client: resolved in commit: [c4caafae4f50f4cf9fe37c4ce64d5806882c5701](#)

6.4 [Medium] Adding liquidity mints LP shares at face value and can dilute existing liquidity providers

File(s): `contracts/pools/PoolBase.sol`

Description: When someone provides liquidity to a pool they receive LP shares, which are a claim on a slice of the pool's two reserves. For that claim to be fair, a deposit should receive shares equal to the fraction of the pool it represents, so a depositor can only ever take back what they put in. This pool mints shares equal to the face value of the deposit, the raw amount handed over, ignoring what a share is currently worth:

```
dy = (dx * reserveY) / reserveX;
// ...
// @audit-issue Shares equal the deposited value, not a fraction of the existing shares.
lpMinted = _normX(dx) + dy;
_mint(msg.sender, lpMinted);
```

Face value is only correct while the pool's total value k (its two reserves) equals its share supply `totalSupply`, so that one share is worth exactly one unit. Once the pool holds more value than that, a face-value deposit hands out shares worth more than the depositor paid. The pool comes to hold that extra value through trading, because when a swap is large enough to move onto the StableSwap curve the trader pays a bit of slippage that stays in the pool. That value belongs to the LPs who are already there, so each existing share becomes worth a little more than one unit ($k / \text{totalSupply}$ rises above 1). Rebalancing does not remove it; it only shuffles value between the two reserves and leaves k unchanged. Withdrawals, meanwhile, always pay a strict fraction of the reserves:

```
amountAssetOut = (lpAmount * reserveX) / totalSupply;
amountUSDZOut = (lpAmount * reserveY) / totalSupply;
```

So a newcomer who deposits at face value and then withdraws at true value walks off with more than they put in, and the difference is taken from the existing LPs.

An example makes the flow concrete. Say the pool starts balanced with only Alice as its liquidity provider. Alice adds a large amount of liquidity, which pushes the pool onto the StableSwap curve, since adding liquidity raises the pool's internal $k/2$ floor until swaps begin pricing on the curve. Bob then makes a swap on that curve and pays slippage, and his lost slippage stays in the pool as extra value; because Alice is the LP, that value is now hers, and her shares are worth more, which is the fair reward for providing the liquidity Bob traded against. Carol wants that value without having earned it, but she cannot add liquidity yet, because the pool's reserve is sitting at its floor. So she first mints some USDZ, which tops the reserve back up and leaves Bob's slippage untouched in the pool, and now that she can add liquidity she deposits, is minted shares at face value (over-credited, since each share is really worth more than one unit), and immediately withdraws, leaving with a share of Bob's slippage. The result is that Carol pockets value that should have gone to Alice and any other LP who was there when Bob traded and her round trip is risk-free.

Recommendation(s): Consider minting LP shares in proportion to the existing share supply so a deposit always receives shares matching its true fraction of the pool, no matter how much extra value the pool has built up.

Status: Fixed

Update from the client: resolved in commit: [4f23167936ab5e285d96e28da70bd80ee472b728](#)

6.5 [Low] Pool swaps have no slippage protection, exposing direct callers to forced slippage

File(s): [contracts/pools/PoolBase.sol](#)

Description: The pool exposes two swap entry points, `swapFromAsset(...)` and `swapFromUSDZ(...)`. Both are external and permissionless, and neither takes a minimum-output or deadline argument, so a caller receives whatever the curve returns at execution time with no floor on the result:

```
function swapFromAsset(uint256 amountIn)
    external override whenNotPaused returns (uint256 amountOut)
{
    // @audit-issue No minAmountOut or deadline, so the caller cannot bound the output.
    // ...
}
```

The only slippage protection in the system lives one layer up, in the router, which reverts when the final output is below the caller's limit:

```
if (amountOut < minAmountOut) revert InsufficientOutput();
```

So router users are protected, but anyone who calls a pool directly has none. This matters because a pool need not be near balance when they trade. Near balance the pool prices at a flat 1:1, but once it is imbalanced it prices on the StableSwap curve and swaps take real slippage, and that imbalanced state can be forced permissionlessly. An attacker can push a target pool onto the StableSwap curve without even holding that pool's assets: they mint USDZ against a different asset's reserve, then swap that USDZ into the target pool to drain it and leave it imbalanced. A direct caller who then swaps against that pool receives a degraded rate, and can be sandwiched, the attacker deepening the imbalance before the victim's swap and unwinding it after, because the pool applies no output check to stop it.

Router users are shielded because the router's `minAmountOut` makes their transaction revert instead, so the exposure is specific to direct pool interaction rather than the protocol's main entry point, which is why this is rated Low.

Recommendation(s): Consider adding a `minAmountOut` (and ideally a deadline) argument to `swapFromAsset(...)` and `swapFromUSDZ(...)` so a direct caller can bound the price they accept, or restricting the pool swap entry points so that only the router can call them. Either approach removes the unprotected direct-swap path.

Status: Fixed

Update from the client: resolved in commit: [e10ffa6097c69835624e82613cb8544c6dd7ce0e](#)

6.6 [Low] Mint and redeem fees are global and cannot be set per asset

File(s): [contracts/StabilizerAMM.sol](#)

Description: The protocol charges a fee on every mint and every redeem. Both fees are single global values set by the admin:

```
uint256 public mintFeeBPS;
uint256 public redeemFeeBPS;
```

Their setters take only a new fee value, with no asset parameter, so one number applies to every collateral at once and there is no way to charge a different mint or redeem fee for a single asset:

```
function setMintFee(uint256 newFeeBPS) external onlyAdmin {
    if (newFeeBPS > MAX_FEE_BPS) revert InvalidFee();
    // @audit-issue One global mint fee for all assets; no per-asset dimension.
    mintFeeBPS = newFeeBPS;
    emit MintFeeUpdated(newFeeBPS);
}
```

A single global fee is fine only while every collateral sits exactly at \$1. In practice the supported stablecoins drift around their peg all the time, and because USDZ is minted and redeemed against them at a fixed 1:1, holding USDZ itself at \$1 requires continuously steering flow per asset: encouraging mints of the assets trading at or above peg, and discouraging mints of, while encouraging redemptions of, any asset trading below it. The correct fee response is therefore per-asset and points in opposite directions on the two paths. On the mint side the fee on a deviating, cheaper asset should be raised while the others stay low, so that bringing weak collateral in for full-value USDZ is discouraged without choking honest minting of the assets holding their peg. On the redeem side it is the reverse: the fee on the deviating asset should be lowered while the others are raised, so that holders are encouraged to redeem the weak asset out of the reserve and discouraged from pulling out the good collateral.

A single global fee for each path cannot express any of this. Raising the global mint fee to slow the intake of a weak asset also taxes minting of every healthy asset, and raising the global redeem fee to protect the good collateral also discourages redeeming the weak asset, which is exactly the action worth encouraging. So the protocol cannot nudge flow toward the assets holding their peg and away from those drifting off it; its only fee lever is a blunt, all-asset one that cannot be aimed where it is needed. A full depeg is just the extreme case of a deviation these fees already cannot steer.

Recommendation(s): Consider making the mint and redeem fees per-asset so that friction can be adjusted on a deviating asset without affecting the others.

Status: Fixed

Update from the client: resolved in commit: [0d5fc98c66696e1f56ced75901a64e534d2c1038](#)

6.7 [Low] The redeem fee accrues as reserve surplus and can be locked below the k/2 floor

File(s): [contracts/StabilizerAMM.sol](#)

Description: The mint fee is booked as claimable revenue: `mintUSDZ(...)` keeps the fee in the StabilizerAMM under `accumulatedFees[asset]` and it is withdrawn through `claimProtocolFees(...)`. The redeem fee is not booked anywhere. `redeemUSDZ(...)` charges the fee in USDZ but burns the full input while paying out only the post-fee asset, so the fee's collateral is never removed from the reserve.

```
uint256 fee = (amountUSDZIn * redeemFeeBPS) / BPS_DENOMINATOR;
amountAssetOut = (amountUSDZIn - fee) / scaleFactor;
// ...
// @audit-issue The full input is burned but only the post-fee asset is paid out, so the
// @audit-issue fee's collateral is left mixed in the reserve, untracked.
IUSDZ(usdz).burn(address(this), amountUSDZIn);
IReserveHolder(reserveHolder).withdrawFromAdapter(msg.sender, asset, amountAssetOut);
```

The fee therefore accrues as over-collateralization, $E = \text{totalReserves}() - \text{backedSupply}()$, mixed in with `reserves[asset]`. The only way to collect it is `rescueExcessCollateral(...)`, which refuses to drop the reserve below the pool's $k/2$ floor.

```
uint256 k = IPool(pool).totalLiquidity();
uint256 postReserve = reserves[asset] - amount;
// ...
// @audit-issue Recovery reverts if it would leave the reserve below the pool's k/2 floor.
if (normPostReserve < k / 2) revert LocalSafetyViolation();
```

The same reserve figure gates liquidity additions: `addLiquidityX(...)` and `addLiquidityY(...)` call `checkLocalSafety(...)`, which passes as long as `reserves[asset]` covers $\text{new}k/2$. Because the accrued fee inflates `reserves[asset]`, a deposit can raise the pool's k until $k/2$ equals the reserve, leaving the fee pinned below the floor and unrecoverable.

For example, on the USDS pool seeded at $100,000 / 100,000$ (reserve at its $100,000$ floor): minting a buffer and redeeming $20,000$ USDZ at a 10% fee leaves $2,000$ of fee as excess (reserve $102,000$, so it is rescuable). Adding the maximum liquidity the reserve allows ($2,000$

per side) makes the pool $102,000 / 102,000$, so $k/2$ becomes $102,000$, exactly the reserve. `rescueExcessCollateral(...)` then reverts for any amount and the $2,000$ fee is stranded. It becomes recoverable again only if that liquidity is removed, which lowers k . The amount at stake is bounded by the accrued fees and no third party gains, so there is no fund loss, but the protocol can be blocked from collecting its own fee revenue.

The same mixing also feeds swap pricing: while the fee sits in `reserves[asset]`, the hybrid swap math counts it as rebalancing capacity, so the pool keeps quoting $1:1$ instead of falling to the StableSwap curve as it should through market activity, pricing against more capacity than its true backing provides.

Recommendation(s): Consider booking the redeem fee in a dedicated accumulator outside `reserves[asset]`, the way the mint fee uses `accumulatedFees`, so it is tracked, claimable through the normal `claimProtocolFees(...)` path, and never subject to a pool's $k/2$ floor.

Status: Fixed

Update from the client: resolved in commit: [0bab64417b8836a756959335fb40ceeb1e78fb9d](#)

6.8 [Low] Missing lock period lets an LP front-run a large swap and capture its fees risk-free

File(s): [contracts/pools/PoolBase.sol](#)

Description: The trading fee from a swap is credited to whoever holds LP shares at the instant of that swap.

```
if (lpFee > 0) {
    feeReserveX += lpFee;
    // @audit Credited by current share of totalSupply, to whoever holds shares now.
    if (totalSupply > 0) accFeePerShareX += (lpFee * ACC_PRECISION) / totalSupply;
}
```

There is no minimum holding period and no lock: `addLiquidityX(...)` and `addLiquidityY(...)` mint shares immediately, and `removeLiquidity(...)` burns them immediately. So anyone who sees a large swap in the mempool can, in the same block, add liquidity just before it, take a proportional share of that swap's fee, then remove the liquidity and claim.

Because the pool is constant-sum, this is risk-free: add and remove are exactly proportional and there is no impermanent loss, so the actor recovers their principal intact and keeps the fee. They supply liquidity for a single block yet earn the same per-share fee as LPs who keep liquidity in the pool, diluting the standing LPs' yield. This is the standard just-in-time liquidity dynamic, worth noting here mainly because the constant-sum design removes the price risk that normally limits it on other AMMs. No protocol or user funds are lost. The effect is a redistribution of fee income away from passive LPs.

Recommendation(s): Consider discouraging just-in-time liquidity, for example by requiring a minimum holding period before freshly added shares accrue fees, or by locking newly minted LP shares for a short window before they can be removed.

Status: Fixed

Update from the client: resolved in commit: [b4127a0d4b64e6a00561715c6454236474a2018b](#)

6.9 [Low] Redeem fees aren't tracked and affect adapter solvency checks

File(s): [contracts/StabilizerAMM.sol](#)

Description: The `StabilizerAMM` implements the functions `mintUSDZ` and `redeemUSDZ` that allow a user to exchange supported stablecoin assets for USDZ, or vice versa. Both functions support a fee mechanism, with the fee amount being limited to a maximum of 10% of the amount being exchanged. The fee mechanism for the mint flow and redeem flow are different in how they track the accumulated fees, and where the fee asset is stored. To describe the different behaviors, let's assume a scenario with a 1% fee on both mint and redeem, with a user who owns 1000 USDC.

For the mint flow, a user calls `mintUSDZ` and provides their 1000 USDC, of which 1% will be treated as mint fees. The 10 USDC in mint fees will stay in custody of the `StabilizerAMM` contract, while the 990 USDC will be transferred to the adapter via a `depositToAdapter` call. The 10 USDC in fees on the AMM contract are accounted for in the `accumulatedFees` mapping, while the 990 USDC in the adapter is tracked in the `reserves` mapping. The user then receives 990 USDZ.

```
function mintUSDZ(address asset, uint256 amountAssetIn) ... {
    // ...
    // Transfer in 1000 USDC
    IERC20(asset).safeTransferFrom(msg.sender, address(this), amountAssetIn);
    // Send out 990 USDC to adapter
    IERC20(asset).forceApprove(reserveHolder, amountAssetIn - fee);
    IReserveHolder(reserveHolder).depositToAdapter(address(this), asset, amountAssetIn - fee);
    // Account for the remaining 10 USDC in 'accumulatedFees'
    accumulatedFees[asset] += fee;
    IUSDZ(usdz).mint(msg.sender, amountUSDZOut);
    // ...
}
```

For the redeem flow, a user calls `redeemUSDZ` and following the prior example, they want to exchange their 990 USDZ for USDC. The fee of 1% will mean that 9USDC (not 10 due to integer division) is treated as fees, and the user is entitled to receive 981 USDC. The 990 USDZ is burned and 981 USDC is withdrawn from the adapter, then passed to the user. The remaining 9 USDC which represent the withdrawal fee is left in the adapter. No updates to the `accumulatedFees` mapping is done and the redeem USDC fee is stored on the adapter which is different to the mint flow which stores the fee asset in the AMM.

```
function redeemUSDZ(address asset, uint256 amountUSDZIn) ... {
    // ...
    // Determine withdrawal amount after fee
    uint256 amountUSDZAfterFee = amountUSDZIn - fee;
    amountAssetOut = amountUSDZAfterFee / scaleFactor;
    // Burn the 990 USDZ
    IReserveHolder(reserveHolder).assertLocalSafety(pool, amountAssetOut);
    IERC20(usdz).safeTransferFrom(msg.sender, address(this), amountUSDZIn);
    IUSDZ(usdz).burn(address(this), amountUSDZIn);
    // Withdraw the 981 (postfee) USDC from the reserve, leaving 9 USDC in reserve as fee
    IReserveHolder(reserveHolder).withdrawFromAdapter(msg.sender, asset, amountAssetOut);
    // ...
}
```

Since the redeem fees remain in the adapter, they are still accounted for in the reserves mapping in the adapter. The only way to collect it is `rescueExcessCollateral(...)`, which refuses to drop the reserve below the pool's $k/2$ floor.

```
uint256 k = IPool(pool).totalLiquidity();
uint256 postReserve = reserves[asset] - amount;
// ...
// @audit-issue Recovery reverts if it would leave the reserve below the pool's k/2 floor.
if (normPostReserve < k / 2) revert LocalSafetyViolation();
```

The same reserve figure gates liquidity additions: `addLiquidityX(...)` and `addLiquidityY(...)` call `checkLocalSafety(...)`, which passes as long as `reserves[asset]` covers $\text{newK}/2$. Because the accrued fee inflates `reserves[asset]`, a deposit can raise the pool's k until $k/2$ equals the reserve, leaving the fee pinned below the floor and unrecoverable.

For example, on a USDS pool seeded at $100,000 / 100,000$ (reserve at its $100,000$ floor): minting a buffer and redeeming $20,000$ USDZ at a 10% fee leaves $2,000$ of fee as excess (reserve $102,000$, so it is rescuable). Adding the maximum liquidity the reserve allows ($2,000$ per side) makes the pool $102,000 / 102,000$, so $k/2$ becomes $102,000$, exactly the reserve. `rescueExcessCollateral(...)` then reverts for any amount and the $2,000$ fee is stranded. It becomes recoverable again only if that liquidity is removed, which lowers k . The amount at stake is bounded by the accrued fees and no third party gains, so there is no fund loss, but the protocol can be blocked from collecting its own fee revenue.

The same mixing also feeds swap pricing: while the fee sits in `reserves[asset]`, the hybrid swap math counts it as rebalancing capacity, so the pool keeps quoting $1:1$ instead of falling to the `StableSwap` curve as it should through market activity, pricing against more capacity than its true backing provides.

Recommendation(s): Consider storing and tracking redeem fees in the same way as mint fees, by withdrawing the entire redeem amount of USDC from the adapter and then using the `accumulatedFees` mapping on redeem.

Status: Fixed

Update from the client: resolved in commit: [0bab64417b8836a756959335fb40ceeb1e78fb9d](#)

6.10 [Info] The AdapterUSDS contract overrides `balance()` to return the live token balance, diverging from the other adapters

File(s): [contracts/adapters/AdapterUSDS.sol](#)

Description: All four adapters inherit `AdapterBase`, whose `balance()` returns the internal `trackedBalance`. Three adapters (`AdapterUSDC`, `AdapterUSDT`, `AdapterPYUSD`) use this default; `AdapterUSDS` alone overrides it to return the live on-chain balance.

```
// AdapterUSDS.sol
function balance() external view override returns (uint256) {
    return IERC20(underlying).balanceOf(address(this));
}
```

```
// AdapterBase.sol
function balance() external view virtual override returns (uint256) {
    return trackedBalance;
}
```

Recommendation(s): Consider making the `balance()` semantics uniform across all adapters, or documenting explicitly why `AdapterUSDS` must expose the live balance while the others expose the tracked balance, and defining a single authoritative balance source to avoid future integrations mixing tracked and live balances.

Status: Fixed**Update from the client:** resolved in commit: [372b95e4e72c9807ecfc6a7cbee739853a90c66d](#)

6.11 [Info] The guardian can halt pool's swaps through the uncapped emergency fee

File(s): [contracts/StabilizerAMM.sol](#)

Description: Two roles can act on a pool. The admin can halt it with `pause()`, which is `onlyAdmin`. The guardian is a separate, lower-privilege role whose power is `setEmergencyDepegFee(...)`, meant to raise a pool's swap fee temporarily during a depeg. Unlike every other fee setter (`setMintFee`, `setRedeemFee`, and `setTradingFee` all cap at 1000 BPS), the emergency fee has no upper bound.

```
function setEmergencyDepegFee(uint256 newFeeBPS) external onlyAMM {
    emergencyDepegFeeBPS = newFeeBPS;
    emit EmergencyDepegFeeSet(newFeeBPS);
}
```

That value becomes the effective fee on every swap ($\text{feeAmount} = \text{amountIn} * \text{getEffectiveFee}() / \text{BPS_DENOMINATOR}$, then $\text{netInput} = \text{amountIn} - \text{feeAmount}$), so an uncapped, guardian-settable fee gives the guardian pausing power as at 10000 BPS (100%) `netInput` is 0 and every swap reverts with `ZeroOutput`; above 10000 the subtraction underflows and reverts. `swapFromAsset(...)`, `swapFromUSDZ(...)`, and every router route through the pool stop working. This is the same effect as `pause()`, reached by the guardian, even though pausing is `onlyAdmin`.

Recommendation(s): Consider capping `emergencyDepegFeeBPS` in the setter in order to prevent the scenario or consider documenting that the guardian should be able to pause.

Status: Acknowledged

Update from the client: acknowledged, guardian already has broad responsibility during a depeg so the halt power was kept; but added a 100% cap as a guard against human error in commit [a55aeca7237aaff32bb88feea62c773d0bea0b8f](#)

6.12 [Info] The AMM and pool pause switches are misaligned in both directions

File(s): [contracts/StabilizerAMM.sol](#), [contracts/pools/PoolBase.sol](#)

Description: The AMM, each pool, USDZ, and the reserve holder each have their own pause switch. Two gaps break the isolation.

First, pausing the AMM stops all pool swaps. Every pool swap ends by calling `rebalance(...)` on the AMM, and `rebalance(...)` is `whenNotPaused`, so pausing the AMM (for example to halt mint/redeem during an incident) makes every `swapFromAsset(...)`, `swapFromUSDZ(...)`, and router swap revert, even though the pools were never paused.

```
// @audit-issue rebalance is whenNotPaused, so an AMM pause reverts every pool swap.
IStabilizerAMM(stabilizerAMM).rebalance(address(this));
```

Second, pausing a pool does not stop rebalancing into it. The user-facing swap functions check the pool's pause, but the AMM-only functions the rebalancer uses do not, and `rebalance(...)` never checks the pool's pause either.

```
// @audit-issue No whenNotPaused, unlike the user-facing swap functions.
function ammSwapFromAsset(...) external override onlyAMM returns (uint256 amountOut) {
    // ...
}
```

Since `rebalance(...)` is permissionless, anyone can call it on a paused pool and move its reserves (pulling collateral in or out). The rebalance only nudges the pool toward balance and pays the caller nothing, so by itself it takes no funds; the concern is that a pool the admin paused (say, to contain a separate bug) is not actually frozen. Pausing the pool's adapter or the AMM does close this path.

Recommendation(s): Consider aligning the two pause switches: do not gate `rebalance(...)` on the AMM pause (or document that an AMM pause is meant to halt swaps too), and gate the AMM-only pool functions or `rebalance(...)` on the target pool's pause so a paused pool is fully frozen.

Status: Fixed**Update from the client:** resolved in commit: [7f88437ce390324a82466fd5152f57b32c30d69f](#)

6.13 [Info] Redeeming a small USDZ amount burns USDZ but returns zero asset for low-decimal collateral

File(s): [contracts/StabilizerAMM.sol](#)

Description: The `redeemUSDZ(...)` function lets a holder burn USDZ (18 decimals) and receive the chosen collateral. For collateral with fewer than 18 decimals (USDC, USDT, PYUSD are 6 on mainnet) the USDZ amount is divided by $10^{18 - \text{assetDecimals}}$ to convert to asset units, and the full input is burned.

```
uint256 scaleFactor = 10 ** (18 - assetDecimals);
uint256 fee = (amountUSDZIn * redeemFeeBPS) / BPS_DENOMINATOR;
uint256 amountUSDZAfterFee = amountUSDZIn - fee;
// @audit-issue For 6-dec assets, any amountUSDZAfterFee < 1e12 floors amountAssetOut to 0.
amountAssetOut = amountUSDZAfterFee / scaleFactor;
// ...
IUSDZ(usdz).burn(address(this), amountUSDZIn);
IReserveHolder(reserveHolder).withdrawFromAdapter(msg.sender, asset, amountAssetOut);
```

There is no check that `amountAssetOut > 0`. If a user redeems any `amountUSDZIn` whose post-fee value is below one asset unit (below `1e12` for a 6-decimal asset), `amountAssetOut` floors to 0, yet the entire `amountUSDZIn` is pulled in and burned and the withdrawal transfers 0 tokens. The user destroys their USDZ and receives nothing; the burned value becomes silent over-collateralization for the protocol.

Recommendation(s): Consider reverting when the computed `amountAssetOut` is zero (as the swap paths already do with `ZeroOutput`), so redemptions that would return nothing fail loudly instead of burning USDZ for no asset.

Status: Fixed

Update from the client: resolved in commit: [0b4d63daed115dc974bacb7a59bd54afeec74a4e](#)

6.14 [Info] Unnecessary transfer before burn during redeem

File(s): [contracts/StabilizerAMM.sol](#)

Description: When redeeming USDZ for an underlying asset in the `StabilizerAMM`, user's funds are transferred from their own address to the AMM contract, and then they are burned. The AMM-only burn function in the USDZ contract allows the AMM to burn funds from any address, not necessarily just funds belonging to itself. This makes the `transferFrom` call to move assets from the caller to the AMM unnecessary, costing additional gas.

```
function redeemUSDZ(...) ... returns (...) {
    // ...
    IERC20(usdz).safeTransferFrom(msg.sender, address(this), amountUSDZIn);
    IUSDZ(usdz).burn(address(this), amountUSDZIn);
    // ...
}
```

Recommendation(s): Consider burning assets directly from the user to reduce gas costs.

Status: Fixed

Update from the client: resolved in commit: [8cf17a173a455fadaa222811486f9c3da72b4ce5](#)

6.15 [Best Practices] The `setGuardian(...)` function emits the wrong value for the old guardian

File(s): [contracts/StabilizerAMM.sol](#)

Description: The `StabilizerAMM` emits `GuardianTransferred(oldGuardian, newGuardian)` when the guardian changes and `AdminTransferred(oldAdmin, newAdmin)` when the admin changes; both share "old value, new value" semantics.

```
function transferAdmin(address newAdmin) external onlyAdmin {
    if (newAdmin == address(0)) revert ZeroAddress();
    emit AdminTransferred(admin, newAdmin);
    admin = newAdmin;
}
```

The `setGuardian(...)` function does the opposite: it overwrites `guardian` first and then emits, reading the already-updated guardian as the "old" argument.

```
function setGuardian(address newGuardian) external onlyAdmin {
    guardian = newGuardian;
    // @audit-issue guardian already equals newGuardian, so old value is emitted as new.
    emit GuardianTransferred(guardian, newGuardian);
}
```

As a result the `GuardianTransferred` event always reports the new guardian in both fields, so off-chain indexers and monitoring can never reconstruct the previous guardian from logs.

Recommendation(s): Consider emitting the `GuardianTransferred` event before the state assignment, matching the ordering already used by `transferAdmin(...)`.

Status: Fixed

Update from the client: resolved in commit: [24bd80ad1c2f1d4e90be3f174115809345e1cfbb](#)

6.16 [Best Practices] Unused function updateReserve

File(s): [contracts/ReserveHolder.sol](#)

Description: The ReserveHolder contract has a function updateReserve which can be used to manually adjust the reserves of a given asset. This function is only accessible to the AMM contract, however, the AMM contract has no logic which calls this function. As a result, the function is uncallable and unused.

```
// Only callable by AMM, but AMM cannot call
function updateReserve(...) external override onlyAMM {
    // ...
}
```

Recommendation(s): Consider removing the unused function.

Status: Fixed

Update from the client: resolved in commit: [e205b51ad87ef2aaa673361cbd314158d4b40d1a](#)

6.17 [Best Practices] Unnecessary double declaration of pool asset

File(s): [contracts/StabilizerAMM.sol](#)

Description: In the function rebalance on the StabilizerAMM the pool address is passed as an argument, from which the pool asset can be derived. This function queries the same pool asset twice, under two different variable names asset and _asset. Since they both hold the same information, one of these declarations can be removed.

```
function rebalance(address pool) external override whenNotPaused noReentrantRebalance {
    address _asset = IPool(pool).asset(); // Asset declaration

    if (poolForAsset[_asset] != pool) revert AssetNotSupported();
    (uint256 delta, bool xGreater) = computeDelta(pool);
    if (delta == 0) return;

    address asset = IPool(pool).asset(); // Asset declaration
    // ...
}
```

Recommendation(s): Consider removing one of these duplicate declarations.

Status: Fixed

Update from the client: resolved in commit: [e16c3260050c61ca5f63e0f45b2de9ce0d755c31](#)

6.18 [Best Practices] The USDZ contract should inherit Openzeppelin ERC20 logic

File(s): [contracts/USDZ.sol](#)

Description: The USDZ contract is an ERC20 token with additional features such as upgradeability, pause/unpause, and the mint/burn flows being accessible to the StabilizerAMM. The underlying ERC20 token logic (such as transfers, allowances, balance checks, etc) has been implemented by the Stabilizer team rather than importing from the OpenZeppelin ERC20 library. A reference implementation should always be imported where possible, especially in the case of heavily standardised EIPs such as ERC20. The custom implementation deviates from the standard in the following ways:

- The transferFrom function emits an Approval event on every call, since it internally calls _approve which emits the event. This excessive event emission when an actual approval didn't occur may affect offchain infrastructure for protocols which integrate with the USDZ token.
- An allowance of uint256.max does not give an infinite allowance. Executing a transferFrom will always reduce the allowance of the caller. While the ERC20 specification doesn't state that uint256.max should be treated as an infinite allowance, it is extremely common in other implementations. Other protocols which integrate with this token may have unexpected behavior when an infinite allowance assumption breaks.
- The errors returned by the USDZ token upon a call failure have a different naming standard, and lack context such as the involved addresses and token amounts. For example: InsufficientBalance vs ERC20InsufficientBalance(sender, balance, needed). Offchain infrastructure, wallets, or block explorers may be designed to detect the more common OpenZeppelin errors and present them in an easier to understand way to the user.

The OpenZeppelin ERC20 library also includes an upgradeable variant [ERC20Upgradeable](#) which supports the USDZ design approach of using a UUPS proxy.

Recommendation(s): Consider reimplementing the USDZ contract to inherit ERC20Upgradeable and then use function overrides to add the additional features such as pausing and AMM-only minting/burning.

Status: Fixed

Update from the client: resolved in commit: [2a103395e6b6fca1d1af99666b306ce0c99dd4f7](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Stabilizer's documentation

The Stabilizer team provided multiple forms of documentation during the audit engagement. A high-level summary of the protocol (available [here](#)) describes each contract and provides a brief explanation of the steps involved in swaps, mints, redemptions, and rebalances. For more detailed information, the technical documentation hosted on GitBook (available [here](#)) describes all the core components of the protocol, including fee mechanisms, liquidity provision, and swaps, in much greater detail. Finally, a whitepaper was also provided that goes into further detail, describing the constant-sum AMM, its invariants, multi-pool stabilization, and liquidity measures.

The codebase itself makes use of NatSpec for some functions, but coverage is sparse and could be improved. It is recommended that each public-facing function have NatSpec documentation. For functions that execute complicated or convoluted logic, inline comments could also be added to improve clarity and help readers understand the code more quickly.

8 Test Suite Evaluation

8.1 Tests Output

```

Ran 17 tests for test/unit/AdapterBase.t.sol:AdapterBaseTest
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 2.06ms (270.25µs CPU time)
Ran 39 tests for test/unit/AccessControl.t.sol:AccessControlTest
Suite result: ok. 39 passed; 0 failed; 0 skipped; finished in 2.28ms (510.50µs CPU time)
Ran 9 tests for test/unit/EmergencyFee.t.sol:EmergencyFeeTest
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 2.31ms (537.54µs CPU time)
Ran 9 tests for test/unit/InitialLiquidity.t.sol:InitialLiquidityTest
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 2.64ms (850.13µs CPU time)
Ran 6 tests for test/integration/Integration_LiquidityFlow.t.sol:IntegrationLiquidityFlowTest
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.65ms (781.46µs CPU time)
Ran 10 tests for test/unit/PPool.t.sol:PPoolTest
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 2.99ms (472.71µs CPU time)
Ran 23 tests for test/unit/AdminTransfer.t.sol:AdminTransferTest
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 3.56ms (717.71µs CPU time)
Ran 12 tests for test/unit/LiquidityRemoval.t.sol:LiquidityRemovalTest
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 3.49ms (935.09µs CPU time)
Ran 10 tests for test/unit/FeeSplit.t.sol:FeeSplitTest
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 3.71ms (1.40ms CPU time)
Ran 7 tests for test/integration/Integration_ProtocolFees.t.sol:IntegrationProtocolFeesTest
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 3.40ms (1.20ms CPU time)
Ran 21 tests for test/unit/ReserveHolder.t.sol:ReserveHolderTest
Suite result: ok. 21 passed; 0 failed; 0 skipped; finished in 4.12ms (1.22ms CPU time)
Ran 21 tests for test/unit/Swap.t.sol:SwapTest
Suite result: ok. 21 passed; 0 failed; 0 skipped; finished in 4.41ms (2.37ms CPU time)
Ran 16 tests for test/unit/Pausability.t.sol:PausabilityTest
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 2.43ms (737.37µs CPU time)
Ran 7 tests for test/integration/Integration_Pausability.t.sol:IntegrationPausabilityTest
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 2.31ms (586.25µs CPU time)
Ran 12 tests for test/integration/Integration_SwapFlow.t.sol:IntegrationSwapFlowTest
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 4.36ms (2.64ms CPU time)
Ran 8 tests for test/integration/Integration_MintRedeemFlow.t.sol:IntegrationMintRedeemFlowTest
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 5.92ms (1.05ms CPU time)
Ran 31 tests for test/unit/StabilizerAMM.t.sol:StabilizerAMMTest
Suite result: ok. 31 passed; 0 failed; 0 skipped; finished in 3.58ms (1.86ms CPU time)
Ran 6 tests for test/integration/Integration_Upgradeability.t.sol:IntegrationUpgradeabilityTest
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 3.49ms (1.80ms CPU time)
Ran 12 tests for test/unit/ProtocolFees.t.sol:ProtocolFeesTest
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 3.04ms (1.35ms CPU time)
Ran 7 tests for test/integration/Integration_MultiPool.t.sol:IntegrationMultiPoolTest
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 12.43ms (11.26ms CPU time)
Ran 11 tests for test/unit/Rebalance.t.sol:RebalanceTest
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 2.20ms (566.33µs CPU time)
Ran 8 tests for test/fuzz/MintRedeem.fuzz.t.sol:MintRedeemFuzzTest
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 15.21ms (44.48ms CPU time)
Ran 6 tests for test/fuzz/Solvency.fuzz.t.sol:SolvencyFuzzTest
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 38.83ms (102.99ms CPU time)
Ran 11 tests for test/fuzz/SwapMath.fuzz.t.sol:SwapMathFuzzTest
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 42.30ms (86.87ms CPU time)
Ran 29 tests for test/unit/PoolBase.t.sol:PoolBaseTest
Suite result: ok. 29 passed; 0 failed; 0 skipped; finished in 32.09ms (1.75ms CPU time)
Ran 2 tests for test/invariant/Solvency.invariant.t.sol:SolvencyInvariantTest
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 39.64ms (60.32ms CPU time)
Ran 5 tests for test/fuzz/LPAccrual.fuzz.t.sol:LPAccrualFuzzTest
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 40.31ms (92.69ms CPU time)
Ran 13 tests for test/unit/Upgradeability.t.sol:UpgradeabilityTest
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 45.29ms (1.30ms CPU time)
Ran 17 tests for test/unit/USDZ.t.sol:USDZTest
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 46.71ms (396.00µs CPU time)
Ran 7 tests for test/fuzz/Amplification.fuzz.t.sol:AmplificationFuzzTest
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 50.55ms (107.71ms CPU time)
Ran 4 tests for test/fuzz/FeeSplit.fuzz.t.sol:FeeSplitFuzzTest
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 69.39ms (33.51ms CPU time)
Ran 4 tests for test/invariant/PoolBalance.invariant.t.sol:PoolBalanceInvariantTest
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 69.26ms (167.77ms CPU time)

Ran 32 test suites in 99.68ms (566.96ms CPU time): 400 tests passed, 0 failed, 0 skipped (400 total tests)

```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.